PatientForm.svelte: Patient registration & edit form component

Project reference: STN22_Thoracic belt for respiratory rate measurement

Authors: Hakkou Dounia

Date: 2025–2026

File header

```
<!--
  Description: Dual-mode Svelte form component for patient registration and editing.
  When the 'patientToEdit' prop is null, the form creates a new patient via POST
/patients.
  When a Patient object is provided, it switches to edit mode and pre-fills all fields,
  then submits via PUT /patients/:id.
  On success, dispatches 'patientAdded' to the parent. On cancel, dispatches 'cancel'.
  All fields are required; a validation guard prevents submission if any are empty.

  Project reference: STN22_Thoracic belt FR
  Authors: Hakkou Dounia
  Date: 2025-2026
  © SIS TEAM NANCY
-->
```

Script block (TypeScript)

```
<script lang="ts">
  // Import patient CRUD helpers and the Patient type from the shared API module
  import { patientsAPI, type Patient } from './api';
  // createEventDispatcher: emits typed custom events to the parent component
  import { createEventDispatcher } from 'svelte';

  // Optional prop: if set, the form operates in edit mode and pre-fills fields.
  // If null (default), the form operates in creation mode.
  export let patientToEdit: Patient | null = null;

  // Typed event dispatcher:
  // - 'patientAdded': emitted after successful create or update, carries the saved
Patient
  // - 'cancel': emitted when the user clicks the Cancel button
  const dispatch = createEventDispatcher<{
    patientAdded: Patient;
    cancel: void;
  }>();

  // Reactive form fields
  // All initialised as empty strings; converted to numbers before API submission.
  let nom = '';
  let prenom = '';
  let taille = '';
  let age = '';
  let sexe: Patient['sexe'] = 'M';  // Default value matches the DB enum
  let telephone = '';
  let poids = '';
  let isSubmitting = false;  // Prevents double submission while request is in flight
  let error = '';            // Displays validation or API error messages in the UI

  // Reactive statement: pre-fill on edit mode
  // The $: prefix makes this block re-run whenever patientToEdit changes.
  // ?? '': guards against null/undefined values in the patient object.
  $: if (patientToEdit) {
    nom       = patientToEdit.nom       ?? '';
    prenom    = patientToEdit.prenom    ?? '';
    taille    = String(patientToEdit.taille    ?? '');
    age       = String(patientToEdit.age       ?? '');
    sexe      = patientToEdit.sexe      ?? 'M';
    telephone = patientToEdit.telephone ?? '';
    poids     = String(patientToEdit.poids     ?? '');
  }

  // resetForm
  // Clears all fields back to their initial empty state.
```

```
  // Called after a successful creation (not after an edit).
  function resetForm() {
    nom = ''; prenom = ''; taille = ''; age = '';
    sexe = 'M'; telephone = ''; poids = '';
  }

  // handleSubmit
  // Called when the form is submitted (on:submit|preventDefault).
  // 1. Validates that all required fields are non-empty.
  // 2. Builds the payload, converting string inputs to numbers where needed.
  // 3. Calls update() in edit mode or create() in creation mode.
  // 4. Dispatches 'patientAdded' with the saved patient on success.
  async function handleSubmit() {
    // Guard: all fields must be filled before sending the request
    if (!nom || !prenom || !taille || !age || !telephone || !poids) {
      error = 'Tous les champs sont obligatoires';
      return;
    }

    isSubmitting = true;  // Disable submit button to prevent double submission
    error = '';           // Clear any previous error message

    try {
      // Build the API payload: convert string form values to typed numbers
      const payload = {
        nom, prenom, sexe, telephone,
        taille: Number(taille),
        age:    Number(age),
        poids:  Number(poids),
      };

      let saved: Patient;

      if (patientToEdit) {
        // Edit mode: PUT /patients/:id, updates the existing record
        saved = await patientsAPI.update(patientToEdit.id, payload);
      } else {
        // Creation mode: POST /patients, creates a new record
        saved = await patientsAPI.create(payload);
      }

      // Notify the parent component that a patient was saved
      dispatch('patientAdded', saved);

      // Reset the form only after a creation; after an edit, keep the data visible
      if (!patientToEdit) resetForm();

    } catch (err: any) {
      // Display the server error message or a generic fallback
      error = err?.message || "Erreur lors de l'enregistrement";
      console.error('Erreur:', err);
    } finally {
      isSubmitting = false;  // Always re-enable the submit button
    }
  }

  // handleCancel
  // Dispatches the 'cancel' event to the parent, which hides the form.
  function handleCancel() {
    dispatch('cancel');
  }
</script>
```

HTML template (Svelte)

```
<div class="patient-form">
  <!-- Title changes dynamically based on mode: creation or edit -->
  <h2>{patientToEdit ? 'Modifier Patient' : 'Enregistrement Patient'}</h2>

  <!-- on:submit|preventDefault: runs handleSubmit without reloading the page -->
  <form on:submit|preventDefault={handleSubmit}>
```

```
    <!-- Row 1: Nom + Prenom (2 columns) -->
    <div class="form-row">
      <div class="form-group">
        <label for="nom">Nom</label>
        <!-- bind:value: two-way binding — updates the 'nom' variable on every keystroke -
->
        <input id="nom" type="text" bind:value={nom} placeholder="Nom" required />
      </div>
      <div class="form-group">
        <label for="prenom">Prenom</label>
        <input id="prenom" type="text" bind:value={prenom} placeholder="Prenom" required
/>
      </div>
    </div>

    <!-- Row 2: Age + Poids + Sexe (3 columns) -->
    <div class="form-row three-cols">
      <div class="form-group">
        <label for="age">Age</label>
        <!-- min/max attributes provide browser-level validation hints -->
        <input id="age" type="number" bind:value={age} min="0" max="150" required />
      </div>
      <div class="form-group">
        <label for="poids">Poids (kg)</label>
        <input id="poids" type="number" bind:value={poids} min="0" max="300" step="0.1"
required />
      </div>
      <div class="form-group">
        <label for="sexe">Sexe</label>
        <!-- <select> with bind:value keeps 'sexe' in sync with the chosen option -->
        <select id="sexe" bind:value={sexe}>
          <option value="M">Masculin</option>
          <option value="F">Feminin</option>
          <option value="Autre">Autre</option>
        </select>
      </div>
    </div>

    <!-- Row 3: Taille + Telephone (2 columns) -->
    <div class="form-row">
      <div class="form-group">
        <label for="taille">Taille (cm)</label>
        <input id="taille" type="number" bind:value={taille} min="0" max="300" step="0.1"
required />
      </div>
      <div class="form-group">
        <label for="telephone">Telephone</label>
        <input id="telephone" type="tel" bind:value={telephone} required />
      </div>
    </div>

    <!-- Error block: rendered only when the 'error' variable is non-empty -->
    {#if error}
      <div class="error-message">{error}</div>
    {/if}

    <div class="button-group">
      <!-- Cancel button: type='button' prevents accidental form submission -->
      <button type="button" class="cancel-btn" on:click={handleCancel}>Annuler</button>

      <!-- Submit button: disabled while a request is in flight to prevent double
submission -->
      <!-- Label changes dynamically: 'Enregistrement...' / 'Modifier' / 'Enregistrer
Patient' -->
      <button type="submit" class="submit-btn" disabled={isSubmitting}>
        {isSubmitting ? 'Enregistrement...' : patientToEdit ? 'Modifier' : 'Enregistrer
Patient'}
      </button>
    </div>
  </form>
</div>
```

CSS styles (scoped)

```
<style>
  /* Scoped styles — only apply to this component's elements */

  /* Main container: white card matching the app design system */
  .patient-form { background: white; padding: 2rem; border-radius: 12px;
    box-shadow: 0 2px 8px rgba(0,0,0,0.1); }

  /* Grid rows: default 2 columns, three-cols variant for the middle row */
  .form-row { display: grid; grid-template-columns: 1fr 1fr; gap: 1rem; }
  .form-row.three-cols { grid-template-columns: 1fr 1fr 1fr; }

  /* Focus state: purple border matching the app primary color */
  input:focus, select:focus { outline: none; border-color: #7c3aed; }

  /* Submit button: purple gradient; disabled state at 60% opacity */
  .submit-btn { background: linear-gradient(135deg, #7c3aed 0%, #93c5fd 100%);
    color: white; }
  .submit-btn:disabled { opacity: 0.6; cursor: not-allowed; }

  /* Error message: red background for high visibility */
  .error-message { padding: 0.75rem; background: #fee2e2;
    color: #dc2626; border-radius: 8px; }

  /* Responsive: single column on small screens */
  @media (max-width: 768px) {
    .form-row, .form-row.three-cols { grid-template-columns: 1fr; }
  }
</style>
```